AGENTIC AI BOOTCAMP

Complete Lecture Notes and Practical Handbook

Prepared and delivered by

Dr Julius Sechang Mboli

mboli4god@gmail.com LinkedIn https://jsmboli.github.io/

AI • LLMs • Agents • Tools • LangChain • LangGraph • Human-in-the-Loop • RAG • Evaluation • Responsible AI

Designed for adult learners and absolute beginners progressing to practical agent development
Bootcamp dates: 19-26 August 2026 | Industrial visit: Amazon fulfilment centre, UK

Learn more at https://jsmboli.github.io/agentic-ai-bootcamp/

For courses at the University of Hull, visit www.hull.ac.uk

To learn more about DAIM, visit https://www.hull.ac.uk/daim

Teaching philosophy - Theory and practical work are deliberately interleaved. Every major concept is introduced in plain language, connected to a mental model, followed by code or a professional scenario, and closed with reflection questions or a practical task.

DAIM MSc Programmes

- MSc Artificial Intelligence and Data Science
- MSc Artificial Intelligence for Business
- MSc Artificial Intelligence for Engineering
- MSc Artificial Intelligence for Healthcare
- MSc AI, Philosophy and Ethics
- MSc Business Analytics and Artificial Intelligence
- MSc Data Science and Artificial Intelligence for Economics

Table of Contents

1 1. How to Use These Notes 1
1.1 Learning outcomes 1
1.2 Recommended learning rhythm 1
2 Artificial Intelligence from First Principles 1
2.1 What is intelligence? 1
2.2 A practical definition of AI 2
2.3 AI, automation and analytics are not identical 2
2.4 Narrow AI and general AI 2
3 Machine Learning, Deep Learning and Neural Networks 3
3.1 What makes machine learning different? 3
3.2 Supervised, unsupervised and reinforcement learning 3
3.3 Neural networks 3
3.4 Why deep learning mattered for language 3
4 Generative AI, Transformers and Large Language Models 4
4.1 What is generative AI? 4
4.2 What is a Large Language Model? 4
4.2.1 Tokens 4
4.2.2 Embeddings 4
4.2.3 Context window 4
4.3 4.The Transformer intuition 4
4.3.1 Self-attention in plain language 4
4.4 Pre-training, instruction tuning and alignment 4
4.5 Temperature and decoding 5
4.6 Hallucination 5
5 Prompting and Reliable Model Interaction 5
5.1 What is a prompt? 5
5.2 A reusable prompt structure 5
5.3 Structured output 5
5.4 Prompt injection 6
6 From LLMs to Agents 6
6.1 Chatbot versus agent 6

6.2 A practical definition of an AI agent 6
6.3 Agency is a spectrum 7
6.4 ReAct 7
6.5 What an agent needs 7
7 Tools, Function Calling and External Action 8
7.1 Why tools matter 8
7.2 Tool anatomy 8
7.3 Read tools versus write tools 8
7.4 Ollama tool calling 8
8 LangChain: The Higher-Level Application Framework 8
8.1 What LangChain is 8
8.2 Why use a framework? 9
8.3 LangChain concepts 9
8.4 Minimal conceptual example 9
9 LangGraph: Stateful Agent Orchestration 9
9.1 Why LangGraph exists 9
9.2 Graph terminology 10
9.3 A first StateGraph 11
9.4 Deterministic workflows versus agentic routing 11
9.5 Common graph patterns 11
10 Human-in-the-Loop (HITL) 11
10.1 Why HITL matters 11
10.2 What genuine HITL looks like in LangGraph 12
10.3 Conceptual interrupt example 12
10.4 Approval policies 12
10.5 HITL failure modes 13
11 State, Memory, Persistence and Context 13
11.1 State is not the same as memory 13
11.2 Short-term and long-term memory 13
11.3 Persistence and checkpoints 13
11.4 11.4 What not to store 13
12 Retrieval-Augmented Generation (RAG) 13
12.1 Why RAG exists 13

12.2 Ingestion pipeline 14
12.3 Chunking 14
12.4 RAG is not a truth machine 14
12.5 RAG versus tools 15
13 Multi-Agent Systems and Workflow Patterns 15
13.1 Why use more than one agent? 15
13.2 Common patterns 15
13.3 Stopping conditions 15
14 LLM Providers: Groq, Ollama and Other Options 15
14.1 Provider abstraction 15
14.2 Groq 16
14.3 Ollama 16
14.4 Other providers 16
14.5 Environment variables 16
15 Evaluation, Observability, Reliability and Safety 17
15.1 What should we evaluate? 17
15.2 Why observability matters 17
15.3 Reliability patterns 17
15.4 Responsible AI 17
15.5 Threats specific to agents 17
16 Amazon Fulfilment-Centre Case Study 18
16.1 Use case 1 - Operational exception triage 18
16.2 Use case 2 - Maintenance knowledge assistant 18
16.3 Use case 3 - Shift handover synthesis 18
16.4 Use case 4 - Training and coaching assistant 19
16.5 Use case 5 - Scenario planning 19
16.6 Observation worksheet for the visit 19
17 Five-Day Bootcamp Teaching Map 19
17.1 Suggested daily teaching rhythm 19
18 Practical Exercises and Mini-Projects 20
18.1 Exercise 1 - Explain the hierarchy 20
18.2 Exercise 2 - Provider comparison 20
18.3 Exercise 3 - Build a safe calculator tool 20

18.4 Exercise 4 - LangGraph workflow 20
18.5 Exercise 5 - HITL 20
18.6 Exercise 6 - RAG 20
18.7 Exercise 7 - Capstone 20
18.8 Capstone marking rubric 21
19 Glossary 21
20 References and Further Reading 22
20.1 Recommended sequence for further reading 22
20.2 Final reflection questions 23
21 Appendix A. Absolute-Beginner Python and Jupyter Primer 24
21.1 A.1 Variables and basic data types 24
21.2 A.2 Lists and dictionaries 24
21.3 A.3 Functions 24
21.4 A.4 Imports and packages 25
21.5 A.5 Exceptions 25
21.6 A.6 What Jupyter actually does 25
21.7 A.7 Notebook discipline 25
22 Appendix B. APIs, HTTP and Environment Variables for Beginners 26
22.1 B.1 What is an API? 26
22.2 B.2 Client and server 26
22.3 B.3 HTTP status codes 26
22.4 B.4 Environment variables 26
22.5 B.5 The common os.environ.get mistake 27
22.6 B.6 .env files 27
23 Appendix C. Provider Setup: Groq and Ollama 27
23.1 C.1 Groq setup 27
23.2 C.2 Ollama setup 28
23.3 C.3 Why an installed model can still “fail” 28
23.4 C.4 Provider metadata students should report 28
24 Appendix D. Debugging Agentic AI Notebooks Systematically 29
24.1 D.1 Use a layer-by-layer diagnostic strategy 29
24.2 D.2 Minimal provider test before frameworks 29
24.3 D.3 Progress indicators 29

24.4 D.4 Timeouts and maximum steps 29
24.5 D.5 Common error matrix 29
25 Appendix E. Reusable Agent Engineering Patterns 30
25.1 E.1 "LLM where useful, code where exact" 30
25.2 E.2 Read-analyse-propose-approve-act 30
25.3 E.3 Evidence before recommendation 30
25.4 E.4 Confidence is not certainty 30
25.5 E.5 Idempotency 30
26 Appendix F. Teaching Questions, Misconceptions and Discussion Prompts 30
26.1 F.1 Common misconceptions to surface explicitly 30
26.2 F.2 Socratic discussion prompts 31
27 Appendix G. A Small End-to-End Architecture Walkthrough 31
27.1 G.1 Proposed state 31
27.2 G.2 Nodes 31
27.3 G.3 Why this is better than one giant prompt 32

1 How to Use These Notes

These notes are written to serve three audiences at the same time: the absolute beginner who needs each idea explained from first principles; the technically curious learner who wants to understand how modern agent systems are assembled; and the practitioner who needs enough implementation detail to build and debug working prototypes.

1.1 Learning outcomes

- Explain AI, machine learning, deep learning, generative AI and LLMs using accurate but accessible language.
- Describe how Transformer-based language models turn text into tokens, process context and generate output.
- Distinguish a chatbot, a deterministic workflow, an LLM-powered workflow and an autonomous or semi-autonomous agent.
- Build tools that an LLM can call and explain why tool use matters.
- Use LangChain components to integrate models, tools and retrieval.
- Use LangGraph to build stateful workflows with explicit nodes, edges, routing, persistence and interrupts.
- Implement Human-in-the-Loop approval, rejection and editing patterns.
- Build a basic RAG system and explain what grounding does and does not solve.
- Compare hosted providers such as Groq with local execution through Ollama.
- Evaluate an agent for correctness, safety, latency, cost and operational usefulness.

1.2 Recommended learning rhythm

Stage	Question to ask	Typical activity
Concept	What is it?	Short lecture and diagram
Mechanism	How does it work?	Walk through process step-by-step
Practice	Can I make it work?	Notebook exercise
Reflection	Why does it matter?	Discuss strengths, limits and risk
Transfer	Where could I use it?	Apply it to the industrial-visit scenario

2 Artificial Intelligence from First Principles

2.1 What is intelligence?

Before defining artificial intelligence, it helps to separate intelligence from consciousness. In engineering, an intelligent system does not need feelings, self-awareness or human-like experience. We normally mean that the system can perform tasks involving perception, prediction, language, planning, decision support, adaptation or problem solving.

2.2 A practical definition of AI

Artificial Intelligence (AI) is the broad field concerned with designing computational systems that perform tasks we associate with intelligent behaviour. These tasks may include recognising images, understanding language, predicting outcomes, recommending actions, solving problems or interacting with an environment.

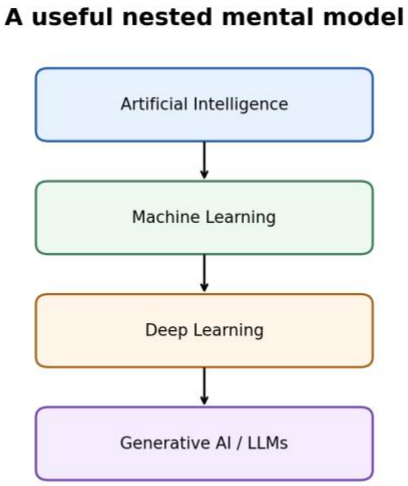


Figure 1. AI is the broad field; machine learning, deep learning and many generative-AI systems can be viewed as progressively more specialised subsets.

2.3 AI, automation and analytics are not identical

Term	Core idea	Example
Automation	Execute predefined rules or procedures	If parcel weight > threshold, route for manual check
Analytics	Summarise or model data to support understanding	Estimate average pick time by zone
Machine learning	Learn statistical patterns from data	Predict probability of late dispatch
Generative AI	Generate new content or structured outputs	Draft a shift handover summary
Agentic AI	Use models, tools, state and control logic to pursue a goal	Investigate a backlog, query data, recommend a response, request approval

2.4 Narrow AI and general AI

Most deployed AI today is narrow: it performs a bounded set of tasks. A warehouse vision model may identify damaged packaging; a language model may summarise maintenance reports. Artificial General Intelligence (AGI) is a broader and contested idea referring to systems with general, flexible capability across many domains. These notes focus on systems that can be engineered and evaluated today, not speculative claims about consciousness or AGI.

Beginner checkpoint - If a system follows fixed rules, it can be automated without being machine learning. If a model predicts from learned data, that is machine learning. If a model generates new text, code or images, it is generative AI. If a system can decide which steps or tools to use in pursuit of a goal, it is moving towards agentic behaviour.

3 Machine Learning, Deep Learning and Neural Networks

3.1 What makes machine learning different?

Traditional programming specifies rules directly: inputs plus rules produce outputs. Machine learning instead learns useful relationships from examples. During training, the model adjusts internal parameters so that its outputs increasingly match desired targets or patterns in the data.

```
# Traditional rule
if temperature > 30:
    action = "increase cooling"

# Machine-learning idea
# Learn a function f(features) -> predicted outcome from historical examples
```

3.2 Supervised, unsupervised and reinforcement learning

Paradigm	Learns from	Typical task	Warehouse illustration
Supervised learning	Labelled examples	Classification/regression	Predict whether an order will miss SLA
Unsupervised learning	Unlabelled structure	Clustering/dimensionality reduction	Group similar process patterns
Reinforcement learning	Rewards from interaction	Sequential decision making	Learn a routing policy in a simulated environment

3.3 Neural networks

A neural network is a parameterised function built from layers of simple mathematical transformations. During training, an optimisation process changes weights to reduce a loss function. Deep learning refers to neural networks with multiple layers that learn increasingly abstract representations.

3.4 Why deep learning mattered for language

Language is sequential, context-dependent and ambiguous. Earlier recurrent neural networks processed sequences step-by-step, but long dependencies and training efficiency were difficult. The Transformer architecture introduced a highly parallel attention-based approach that became foundational for modern LLMs [1].

4 Generative AI, Transformers and Large Language Models

4.1 What is generative AI?

Generative AI models learn patterns in data and produce new outputs that resemble the structure of their training data. Depending on the model, outputs can include text, images, audio, video, code, structured JSON or combinations of these modalities.

4.2 What is a Large Language Model?

An LLM is a large neural language model trained to predict or generate sequences of tokens. It does not retrieve a single stored sentence from a database. Instead, it computes probabilities for possible next tokens conditioned on the context it has been given.

4.2.1 Tokens

A token is a unit processed by a language model. A token may be a whole word, part of a word, punctuation or another text fragment. Tokenisation allows a model to work with a finite vocabulary while representing a very large range of text.

4.2.2 Embeddings

An embedding represents a token, sentence or document as a vector of numbers. The position of vectors in a learned space can encode semantic relationships. Embeddings are central to retrieval systems because semantically related text can be found using vector similarity.

4.2.3 Context window

The context window is the amount of tokenised information the model can consider in a single interaction. A larger context window allows more instructions, conversation history or documents, but longer prompts can increase latency and cost and may still lead to attention dilution. Ollama documentation, for example, treats context length as an explicit runtime consideration for agent and coding workloads [14].

4.3 4.The Transformer intuition

The Transformer architecture uses attention to compute which parts of the input should influence one another. Instead of reading a sentence only from left to right through a recurrent state, self-attention allows representations to incorporate information from other positions in the sequence. The 2017 Transformer paper demonstrated that an architecture based on attention could outperform previous sequence models while being more parallelisable [1].

4.3.1 Self-attention in plain language

Imagine the sentence “The robot moved the parcel because it was blocking the aisle.” To interpret “it”, a model needs to identify which earlier concept is relevant. Attention mechanisms learn weighted relationships between positions so that the representation of each token can incorporate context from other tokens.

4.4 Pre-training, instruction tuning and alignment

1. Pre-training: the model learns broad statistical regularities from very large corpora.

2. Instruction tuning: the model is trained on examples where following an instruction is rewarded.

3. Preference/alignment stages: feedback or preference data can be used to make outputs more helpful, safer or better aligned to desired behaviours.

4. Inference: the already-trained model is prompted and generates output; this is what students do when calling Groq or Ollama.

4.5 Temperature and decoding

Generation involves choosing among possible next tokens. Lower temperature generally makes output more conservative and repeatable; higher temperature allows more variation. Temperature does not make a model "more intelligent". For deterministic business workflows, lower variability is often preferable, especially when output is parsed by software.

4.6 Hallucination

A hallucination is a fluent output that is unsupported, incorrect or fabricated relative to the task or evidence. Hallucination is not a separate "bug" that can simply be turned off; it emerges from probabilistic generation, incomplete grounding, ambiguous instructions and model limitations. Tools, retrieval, validation and HITL are ways to reduce risk, not guarantees of truth.

5 Prompting and Reliable Model Interaction

5.1 What is a prompt?

A prompt is the information supplied to a model to guide its behaviour. In chat systems, prompts are typically structured as messages with roles such as system, user, assistant and tool. Good prompting is less about magic phrases and more about specifying task, context, constraints, desired output and evaluation criteria.

5.2 A reusable prompt structure

Component	Purpose	Example
Role/context	Sets relevant perspective	You are assisting an operations analyst.
Task	States what must be achieved	Identify three plausible causes of the backlog.
Evidence	Provides facts to use	Use only the metrics supplied below.
Constraints	Limits unacceptable behaviour	Do not recommend changes to safety rules.
Output format	Makes downstream use easier	Return JSON with cause, evidence, confidence.
Quality criteria	Defines what "good" means	Each recommendation must cite a metric.

5.3 Structured output

When software needs to consume an LLM response, free-form prose is fragile. Structured output constrains the response to a schema such as JSON. Ollama supports structured outputs through JSON schemas [16], while many hosted providers offer equivalent schema or tool interfaces.

```
from pydantic import BaseModel

class Finding(BaseModel):
    issue: str
    evidence: str
    confidence: float

# Ask the model/provider to return data matching this schema.
```

5.4 Prompt injection

Prompt injection occurs when untrusted content attempts to override the system's intended instructions. In RAG or tool-enabled systems, retrieved documents, webpages and tool outputs may contain adversarial instructions. Treat external content as data, not authority. Keep sensitive actions behind explicit policies and HITL.

6 From LLMs to Agents

6.1 Chatbot versus agent

System	Chooses actions?	Uses tools?	Has persistent state?	Typical example
Simple chatbot	Usually no	Optional	Usually limited	Question answering
LLM workflow	Partly predetermined	Often	Can	Summarise → classify → draft
Agent	Yes, within constraints	Yes	Often	Investigate a task and decide next step
Multi-agent system	Several specialised decision-makers	Yes	Often	Planner + researcher + reviewer

6.2 A practical definition of an AI agent

For this bootcamp, an AI agent is a software system in which a model participates in deciding what action should happen next in pursuit of a goal, potentially using tools, state, memory and feedback. The crucial difference from ordinary chat is not the conversational interface; it is the ability to select or sequence actions.

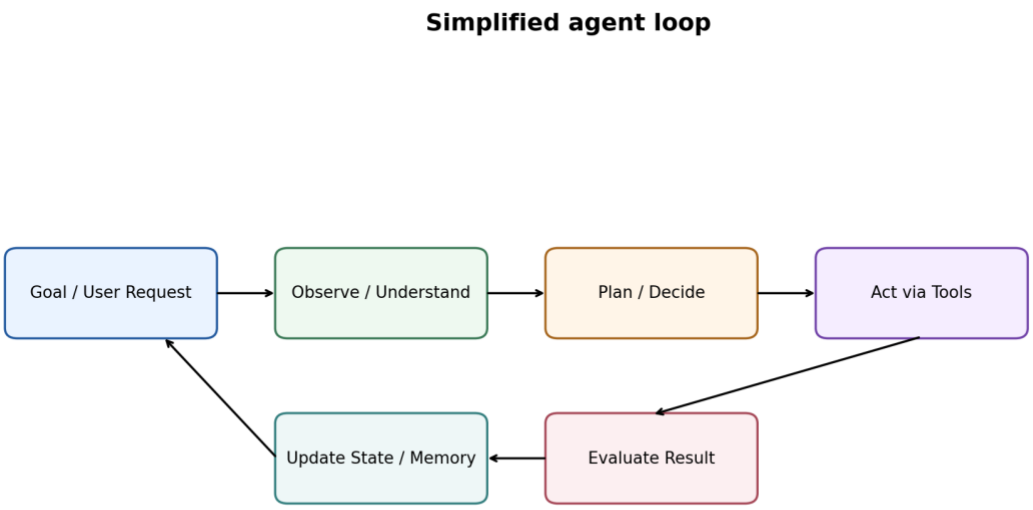


Figure 2. A simplified observe-decide-act-evaluate loop. Real systems usually add policies, limits and human oversight.

6.3 Agency is a spectrum

"Agentic" should not be treated as a binary label. A highly controlled workflow may allow the model to choose only between two safe tools. A more autonomous research agent may dynamically plan many steps. Production systems should grant only the level of autonomy that is justified by task value and risk.

6.4 ReAct

ReAct is an influential agent pattern that interleaves reasoning and acting so that a model can use observations from external actions to update its approach. The original work demonstrated gains on question-answering and interactive tasks by combining these two capabilities [2]. Modern agent frameworks often implement variants of this loop, although production systems should avoid exposing private chain-of-thought and should instead log concise action rationales, tool calls and outcomes.

6.5 What an agent needs

- A goal or request.
- A model capable of interpreting the request and deciding next steps.
- Tools that expose safe, typed capabilities.
- State that records what has happened.
- Policies and constraints that limit unsafe actions.
- Stopping conditions so the agent does not loop indefinitely.
- Observability so humans can inspect what happened.
- Human approval where consequences justify oversight.

7 Tools, Function Calling and External Action

7.1 Why tools matter

An LLM is good at language generation but is not inherently a calculator, database, web browser, inventory system or actuator. Tools bridge this gap by exposing ordinary software functions that the model can request. LangChain describes tools as callable functions with defined inputs and outputs that allow agents to fetch data, execute code, query databases or take actions [7]. Toolformer similarly demonstrated the broader idea that language models can learn when and how to invoke external APIs [3].

7.2 Tool anatomy

```
def calculate_backlog(arrivals_per_hour: float, processed_per_hour: float) -> float:
    """Return the net backlog growth per hour."""
    return arrivals_per_hour - processed_per_hour
```

A production tool should have a clear name, a concise description, explicit parameter types, validated inputs, deterministic behaviour where possible, controlled permissions and meaningful errors. Tool descriptions matter because the model uses them to decide when a tool is relevant.

7.3 Read tools versus write tools

Tool type	Risk	Example	Recommended control
Read-only	Usually lower	Read queue length	Authentication + logging
Analytical	Low to medium	Calculate staffing scenario	Validated inputs + limits
Write/change	Medium to high	Update workflow configuration	Approval + audit log
External side effect	High	Send message, place order, control machine	Strong auth + HITL + idempotency

7.4 Ollama tool calling

Ollama supports tool/function calling for models that implement the capability, allowing a model to request a tool and incorporate the result into its response [13]. Local execution is attractive for privacy and cost control, but model size and capability strongly affect reliability.

8 LangChain: The Higher-Level Application Framework

8.1 What LangChain is

LangChain is an open-source framework for building agents and LLM-powered applications from interoperable components. In the current ecosystem, it provides higher-level APIs for model integrations, tools, agents, retrieval and middleware, while LangGraph supplies the lower-level stateful runtime used when greater orchestration control is required [4, 6].

8.2 Why use a framework?

- Provider abstraction: swap or compare model providers more easily.
- Tool abstraction: describe Python functions so models can use them.
- Message abstractions: represent user, assistant and tool interactions consistently.
- Retrieval integration: connect embeddings, vector stores and retrievers.
- Agent abstractions: assemble common model-tool loops quickly.
- Middleware: apply policies such as HITL to sensitive calls.

8.3 LangChain concepts

Concept	Beginner interpretation
Chat model	A standard interface to a conversational LLM provider
Message	A role-labelled item in the conversation
Tool	A callable capability exposed to a model
Retriever	A component that returns relevant documents
Agent	A loop in which a model can choose actions/tools
Middleware	Logic wrapped around agent activity for control or monitoring

8.4 Minimal conceptual example

```
# Conceptual LangChain-style example
from langchain_core.tools import tool

@tool
def get_zone_backlog(zone: str) -> int:
    """Return the current backlog count for a fulfilment zone."""
    return 127

# A compatible chat model can be bound to tools,
# then an agent loop decides whether to call them.
```

Important distinction - LangChain is not “the AI model”. It is application glue and higher-level engineering abstractions around models, tools, data and workflows.

9 LangGraph: Stateful Agent Orchestration

9.1 Why LangGraph exists

Once a workflow has branches, loops, retries, human approval, persistence and long-running state, a simple sequential chain becomes difficult to reason about. LangGraph is a low-level orchestration framework and runtime focused on durable execution, streaming and human-in-the-loop behaviour [5, 6]. LangChain agents themselves can run on top of LangGraph.

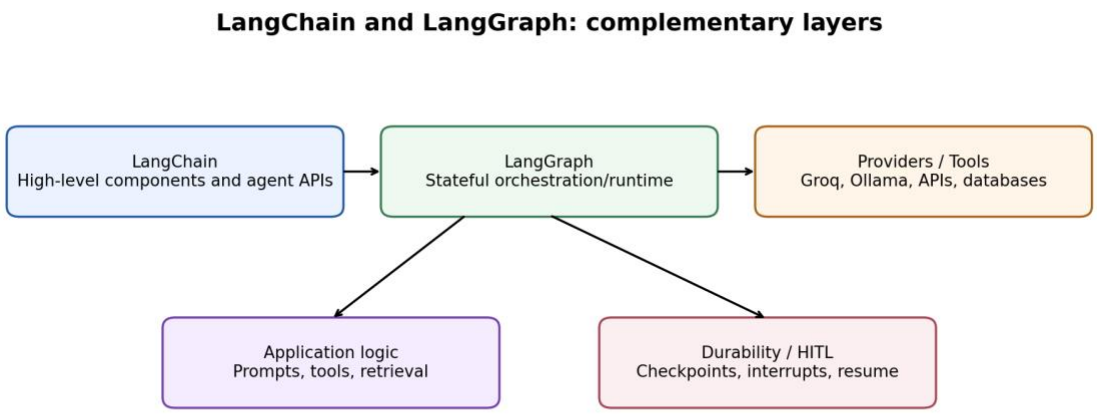


Figure 3. A practical mental model: LangChain offers higher-level application components; LangGraph offers explicit stateful orchestration and runtime capabilities.

9.2 Graph terminology

Term	Meaning
State	The shared data carried through the workflow
Node	A unit of work that reads and/or updates state
Edge	A transition from one node to another
Conditional edge	A transition selected according to logic or model output
START/END	Special entry and termination points
Checkpointer	Persistence mechanism used to save state between steps
Thread	Identifier used to continue a specific persisted interaction

9.3 A first StateGraph

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    question: str
    answer: str

def analyse(state: State):
    return {"answer": f"Analysed: {state['question']}"}

builder = StateGraph(State)
builder.add_node("analyse", analyse)
builder.add_edge(START, "analyse")
builder.add_edge("analyse", END)
graph = builder.compile()

result = graph.invoke({"question": "Why is the backlog rising?", "answer": ""})
```

9.4 Deterministic workflows versus agentic routing

A graph can be fully deterministic, fully model-routed, or hybrid. For operational systems, hybrid designs are often easier to govern: deterministic code handles known business rules and safety constraints, while the model handles interpretation, synthesis or bounded decisions.

9.5 Common graph patterns

Pattern	Purpose	Example
Prompt chain	Sequential transformations	Extract facts → classify → summarise
Routing	Choose specialised branch	Safety issue → safety workflow
Parallelisation	Run independent work concurrently	Retrieve policy + metrics + maintenance data
Orchestrator-worker	Delegate subtasks	Planner delegates analyses
Evaluator-optimizer	Generate, evaluate, refine	Draft plan until criteria met
HITL interrupt	Pause for human decision	Approve high-impact action

LangGraph documentation describes workflows such as evaluator-optimizer loops where generation can be repeated based on evaluator or human feedback [10].

10 Human-in-the-Loop (HITL)

10.1 Why HITL matters

An AI system can be technically capable but operationally unsuitable if it acts without appropriate oversight. HITL places a human decision at selected points where judgement, accountability or risk justifies intervention. The goal is not to make every step manual; it is to put human authority at the right control points.

10.2 What genuine HITL looks like in LangGraph

LangGraph interrupts allow graph execution to pause, persist state and wait for external input. When resumed, the workflow continues from the saved state. This mechanism supports real approval, editing or rejection rather than merely asking the LLM to imagine that a human approved something [8, 9].

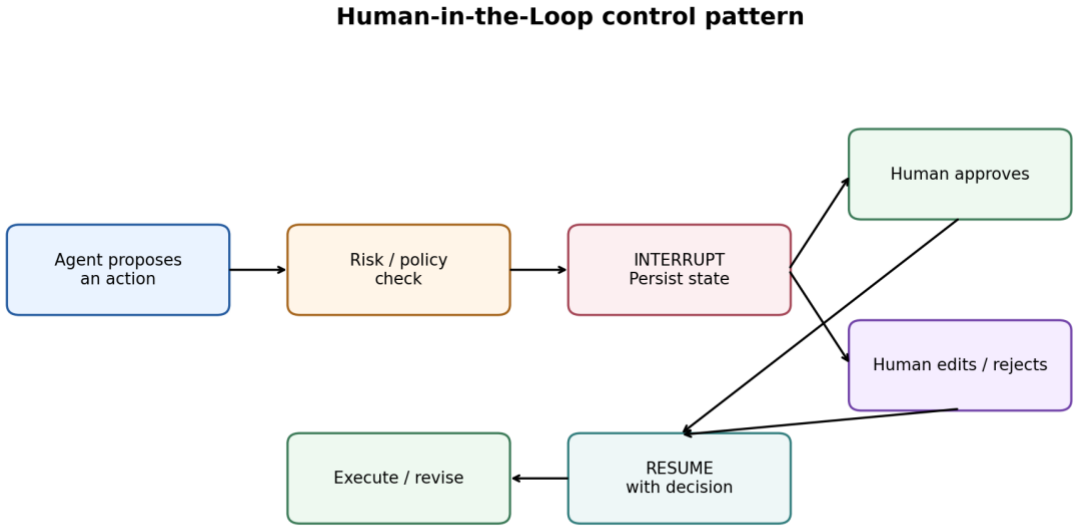


Figure 4. A genuine HITL flow pauses execution before a consequential step, records state, obtains an external human decision and resumes.

10.3 Conceptual interrupt example

```
from langgraph.types import interrupt, Command

def approval_node(state):
    decision = interrupt({
        "proposal": state["proposal"],
        "risk": state["risk"],
        "question": "Approve, edit or reject this action?"
    })
    return {"human_decision": decision}

# Later, using the same persisted thread:
# graph.invoke(Command(resume={"action": "approve"}), config=config)
```

10.4 Approval policies

Action	Default policy	Why
Read public reference data	No approval	Low consequence
Run calculator	No approval	Deterministic and reversible
Draft an email	Review before sending	External communication
Change staffing allocation	Supervisor approval	Operational consequence
Disable safety alert	Never autonomous	Potentially severe harm

10.5 HITL failure modes

- Approval fatigue: humans click “approve” without reading.
- Poor context: reviewer cannot see evidence used by the agent.
- Wrong review point: approval occurs after the irreversible action.
- No audit trail: later reviewers cannot reconstruct who approved what.
- Over-trust: humans assume the system has checked facts it never checked.
- Ambiguous authority: responsibility between agent and human is unclear.

11 State, Memory, Persistence and Context

11.1 State is not the same as memory

State is the data carried through a particular workflow execution. Memory refers more broadly to information retained from previous interactions and made available later. A graph may have rich state without long-term memory, and an application may maintain memory outside the graph in a database.

11.2 Short-term and long-term memory

LangChain conceptual documentation distinguishes memory by recall scope, including short-term conversational state and longer-lived information that can be recalled across interactions [11].

Type	Typical lifetime	Example
Working state	One run or thread	Current task, tool outputs, risk score
Conversation memory	Several turns	What the student previously asked
Long-term memory	Across sessions	User preferences or recurring facts
External system of record	Authoritative business lifetime	Inventory database, HR system

11.3 Persistence and checkpoints

Persistence is essential when workflows may pause, fail or continue later. LangGraph checkpointers support resumption and HITL by retaining graph state so the application can inspect and continue a thread [9].

11.411.4 What not to store

Do not automatically treat every conversation as useful memory. Retention should be purposeful, minimised, access-controlled and consistent with privacy requirements. Sensitive personal data, credentials and temporary secrets should not be placed in casual agent memory.

12 Retrieval-Augmented Generation (RAG)

12.1 Why RAG exists

A model’s internal parameters are not a reliable database of current organisational facts. RAG retrieves relevant information at query time and supplies it as context to the model. This can improve grounding, allow private-domain knowledge to be used without retraining the model, and make sources inspectable.

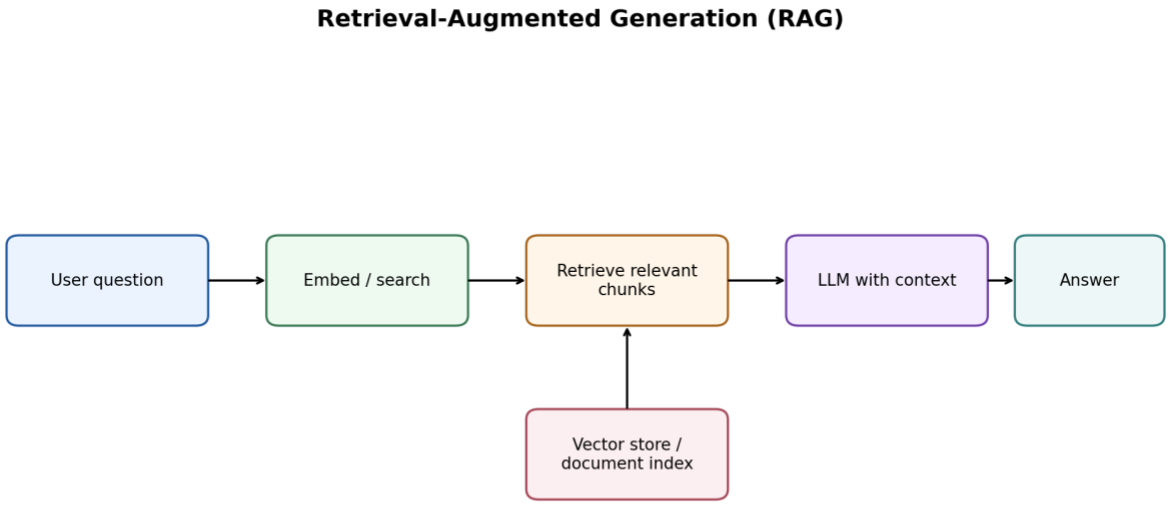


Figure 5. Basic RAG pipeline: retrieve relevant material and place it in the model context before generation.

12.2 Ingestion pipeline

1. Load documents.

2. Clean and split documents into chunks.

3. Create embeddings for chunks.

4. Store embeddings and metadata in an index or vector store.

5. At query time, embed/search the question.

6. Retrieve the most relevant chunks.

7. Construct a prompt with retrieved evidence.

8. Generate an answer and, ideally, include source citations.

12.3 Chunking

Chunk size is a design trade-off. Very small chunks can lose context; very large chunks may dilute relevance and waste context window. Overlap can preserve continuity but increases index size. Chunking should reflect document structure where possible—sections, paragraphs, procedures or records—not arbitrary character counts alone.

12.4 RAG is not a truth machine

- Retrieval can return irrelevant or outdated documents.
- The model can still misinterpret correct evidence.
- Conflicting documents require resolution logic.
- Access controls must apply before retrieval, not only after generation.
- Retrieved text can contain prompt injection or malicious instructions.

12.5 RAG versus tools

Need	RAG	Tool/API
Find policy paragraph	Excellent	Possible but unnecessary
Get live queue length	Poor if index is stale	Excellent
Perform exact calculation	Not appropriate	Calculator/tool
Explain historical manual	Good	Optional
Change business record	Never	Controlled write tool

13 Multi-Agent Systems and Workflow Patterns

13.1 Why use more than one agent?

Specialised agents can divide a complex problem into bounded roles: for example, one agent retrieves operational evidence, another evaluates safety constraints and a third produces a final synthesis. However, multi-agent design adds communication overhead, latency, cost and failure modes. Use it when decomposition creates real value, not merely because “multi-agent” sounds advanced.

13.2 Common patterns

Pattern	Description	Risk
Supervisor-workers	One coordinator delegates to specialists	Supervisor becomes bottleneck
Peer collaboration	Agents exchange findings	Long loops and duplicated work
Debate / critic	One proposes, another critiques	Can create verbosity without better accuracy
Pipeline specialists	Each agent handles one stage	Errors propagate downstream
Human-supervised team	Human approves key transitions	Slower but more governable

13.3 Stopping conditions

Every agent loop should have an explicit reason to stop: goal achieved, maximum steps reached, confidence threshold met, no useful next action, user intervention required, or policy violation encountered. “Continue until the model feels finished” is not a robust engineering control.

14 LLM Providers: Groq, Ollama and Other Options

14.1 Provider abstraction

A provider is the service or runtime that executes a model. Separating provider logic from application logic makes teaching more inclusive and systems more portable. The same conceptual agent can often run with a hosted provider or a local model, although capabilities such as tool calling, structured output, multimodality and context length differ.

14.2 Groq

Groq provides hosted inference with a Python SDK and a catalogue of supported models. As of August 2026, its documentation lists qwen/qwen3.6-27b among supported models and describes it as an agentic coding and reasoning model [12]. Model availability changes, so production code should not hard-code an assumption that a particular model will exist indefinitely.

```
import os
from groq import Groq

client = Groq(api_key=os.environ["GROQ_API_KEY"])
response = client.chat.completions.create(
    model="qwen/qwen3.6-27b",
    messages=[{"role": "user", "content": "Explain agentic AI in simple terms."}],
)
print(response.choices[0].message.content)
```

14.3 Ollama

Ollama provides a local runtime and API for running models on a student's own computer. Its API is automatically available locally when Ollama is running, and its official documentation provides Python and JavaScript libraries [15]. It supports chat, tool calling, structured outputs, thinking controls and OpenAI-compatible interfaces for supported models [13-17].

```
from ollama import chat

response = chat(
    model="qwen3.5:0.8b",
    messages=[{"role": "user", "content": "What is HITL?"}],
    think=False,
)
print(response.message.content)
```

14.4 Other providers

The same principles apply to OpenAI, Google Gemini, Anthropic Claude, DeepSeek and other providers: keep credentials in environment variables, validate model capabilities, respect rate limits, log provider/model metadata and avoid coupling business logic to a single vendor unless there is a deliberate reason.

14.5 Environment variables

```
# .env.example  - never commit real secrets
GROQ_API_KEY=replace_me
OPENAI_API_KEY=replace_me
GOOGLE_API_KEY=replace_me
ANTHROPIC_API_KEY=replace_me
DEEPSEEK_API_KEY=replace_me
OLLAMA_HOST=http://127.0.0.1:11434
```

Security - An API key is a credential. Do not paste live keys into notebooks, GitHub repositories, screenshots or lecture notes. Rotate any key that has been exposed.

15 Evaluation, Observability, Reliability and Safety

15.1 What should we evaluate?

Dimension	Question	Possible metric
Task success	Did the agent achieve the goal?	Success rate
Correctness	Were facts and calculations right?	Exact match / reviewer score
Grounding	Are claims supported by evidence?	Citation precision
Tool use	Were correct tools selected?	Tool-selection accuracy
Safety	Did it avoid prohibited actions?	Policy violation rate
Latency	How long did users wait?	p50/p95 response time
Cost	What did the run consume?	Tokens / API cost
Human burden	How much review was needed?	Approval rate / review time

15.2 Why observability matters

When an agent fails, the final answer alone is not enough to debug it. Useful traces record provider, model, timestamps, node transitions, tool name, validated arguments, tool outputs, errors, retries, token usage and human decisions. This is why the bootcamp notebooks report model/provider metadata and show progress rather than leaving learners staring at a running cell.

15.3 Reliability patterns

- Validate model output with schemas.
- Use deterministic code for arithmetic and business rules.
- Set timeouts and maximum iteration counts.
- Retry transient errors with backoff, but not invalid requests indefinitely.
- Use idempotency for side-effecting tools.
- Prefer read-only tools during early prototyping.
- Keep high-risk actions behind HITL.
- Test with adversarial and ambiguous inputs.

15.4 Responsible AI

Responsible agent engineering requires attention to validity, reliability, safety, security, privacy, transparency, explainability, fairness and accountability. NIST's AI Risk Management Framework and its Generative AI Profile provide structured guidance for identifying and managing AI risks [18, 19].

15.5 Threats specific to agents

- Prompt injection through retrieved or tool-supplied content.
- Excessive tool permissions.
- Data exfiltration through external tools.
- Unsafe automated actions.

- Infinite or expensive loops.
- Model/tool mismatch and malformed arguments.
- Hidden stale state or memory poisoning.
- Human over-reliance on confident outputs.

16 Amazon Fulfilment-Centre Case Study

The industrial visit provides a useful lens for converting abstract agent concepts into operational questions. The examples in this section are illustrative teaching scenarios, not claims about the specific systems deployed at the site visited.

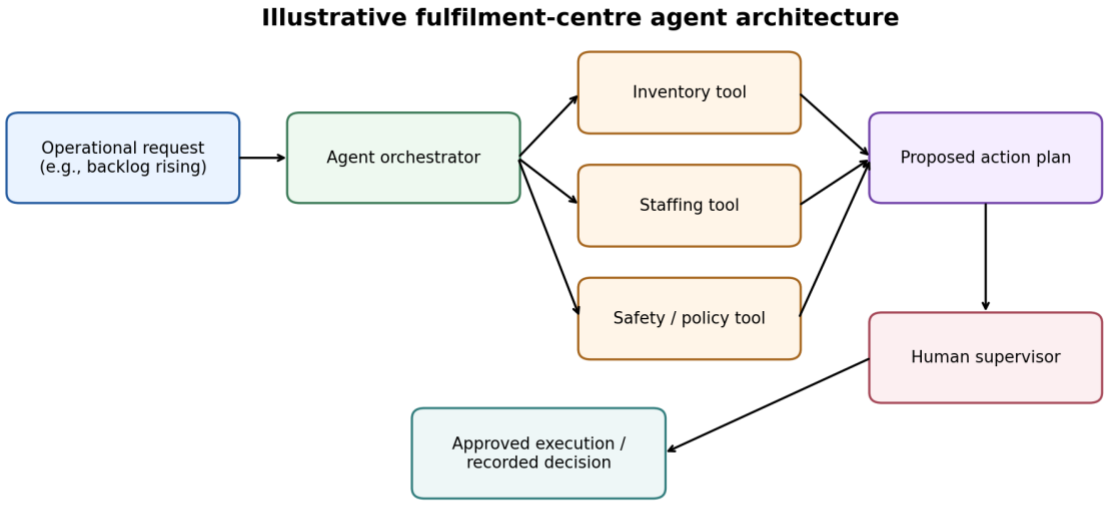


Figure 6. Illustrative semi-autonomous operations-support agent with multiple read/analysis tools and a supervisor approval point.

16.1 Use case 1 - Operational exception triage

Goal: investigate why a process area is developing a backlog. The agent might read non-sensitive queue metrics, compare current versus baseline performance, retrieve relevant operating guidance, calculate throughput gaps and produce a ranked list of plausible causes. It should not autonomously change safety rules or staffing assignments.

16.2 Use case 2 - Maintenance knowledge assistant

Goal: help an authorised technician find relevant maintenance procedures and prior incident summaries. RAG can retrieve documentation while tools can access live machine status. Human technicians retain responsibility for physical intervention.

16.3 Use case 3 - Shift handover synthesis

Goal: summarise structured metrics, open issues and approved notes into a concise handover. A deterministic data pipeline can collect facts; the model can turn them into readable language; a supervisor can review before publication.

16.4 Use case 4 - Training and coaching assistant

Goal: answer questions about standard procedures using approved training material. RAG supports grounding, while access control prevents retrieval of materials the user should not see.

16.5 Use case 5 - Scenario planning

Goal: compare "what-if" scenarios such as changes in arrival rate or processing capacity. The LLM should not perform arithmetic mentally when a calculator or optimisation tool can provide exact results.

16.6 Observation worksheet for the visit

Observe	Ask yourself	Agentic AI connection
Where decisions are made	What information does a human need?	Potential agent state/tools
Where exceptions occur	What triggers escalation?	Routing and HITL
Where data is displayed	Is it live, historical or predictive?	Tool/RAG choice
Where safety gates exist	Can software bypass them?	Policy boundaries
Where handovers occur	What information is repeatedly summarised?	LLM synthesis
Where repetitive searches happen	Are people locating procedures manually?	RAG opportunity

17 Five-Day Bootcamp Teaching Map

Teaching day	Core theory	Practical emphasis	Outcome
Day 1	AI → ML → DL → GenAI → LLMs; prompting	Provider setup and first calls	Students can explain and call an LLM
Day 2	Agents, tools, ReAct, LangChain	Build tools and a tool-using agent	Students understand action selection
Industrial visit	Observe operational decisions and controls	Structured observation worksheet	Students identify grounded use cases
Day 3	LangGraph, state, routing, persistence	Build explicit StateGraph workflow	Students build inspectable orchestration
Day 4	RAG, memory, HITL	Retrieve evidence and pause for approval	Students combine grounding and oversight
Day 5	Multi-agent, evaluation, responsible AI	Capstone prototype and presentation	Students justify architecture and controls

17.1 Suggested daily teaching rhythm

1. Concept lecture (20-30 min).

2. Live code demonstration (15-20 min).

3. Guided student practical (30-45 min).

4. Short debrief and misconception check (10-15 min).

5. Second theory block introducing the next component.

6. Applied mini-project using the new component.

7. End-of-day reflection: what worked, what failed, what would change in production?

18 Practical Exercises and Mini-Projects

18.1 Exercise 1 - Explain the hierarchy

Task: in pairs, explain the relationship between AI, machine learning, deep learning, generative AI, LLMs and agents without using jargon. Then produce one real example for each term.

18.2 Exercise 2 - Provider comparison

Run the same bounded prompt through Groq and an Ollama model. Record provider, model, latency, token counts if available, output quality and whether the model followed the requested format. Discuss why “fastest” is not automatically “best”.

18.3 Exercise 3 - Build a safe calculator tool

```
def throughput_gap(incoming_per_hour: float, processed_per_hour: float) -> float:
    if incoming_per_hour < 0 or processed_per_hour < 0:
        raise ValueError("Rates must be non-negative")
    return incoming_per_hour - processed_per_hour
```

Extend the tool with type validation, clear documentation and tests. Ask the agent to decide when the calculator is necessary rather than doing arithmetic in prose.

18.4 Exercise 4 - LangGraph workflow

Build a graph with nodes for classify_request → gather_evidence → analyse → draft_response. Add conditional routing so a safety-related request goes to a safety_review node.

18.5 Exercise 5 - HITL

Modify the graph so any proposed action marked medium or high risk triggers an interrupt. The reviewer must be able to approve, reject or edit. Persist the thread so the graph can resume.

18.6 Exercise 6 - RAG

Index a small set of approved policy or training documents. Ask questions whose answers are and are not present in the documents. Require the system to say when evidence is insufficient.

18.7 Exercise 7 - Capstone

Design a semi-autonomous operations-support agent inspired by the industrial visit. Your architecture must contain at least three tools, one retrieval source, explicit graph state, one conditional route, one HITL checkpoint, one stopping condition and an evaluation plan. The goal is not to automate a real facility; it is to demonstrate robust architecture thinking.

18.8 Capstone marking rubric

Criterion	Weight	Evidence expected
Problem definition	15%	Clear user, goal, boundaries
Architecture	20%	Justified nodes/tools/state
Working implementation	20%	Reproducible notebook/demo
HITL and safety	15%	Meaningful approval boundary
Grounding/retrieval	10%	Relevant evidence handling
Evaluation	10%	Metrics and failure tests
Communication	10%	Clear explanation and reflection

19 Glossary

Term	Meaning
Agent	A system in which a model can participate in selecting actions in pursuit of a goal.
API	Application Programming Interface; a defined way for software systems to communicate.
Checkpoint	Persisted snapshot/state allowing a workflow to resume.
Context window	Maximum tokenised context available to a model for a request.
Embedding	Vector representation used to capture semantic relationships.
Hallucination	Unsupported or incorrect generated content presented fluently.
HITL	Human-in-the-Loop; human oversight inserted at selected points in an AI process.
Inference	Running a trained model to produce outputs.
LangChain	Higher-level framework for LLM applications, model integrations, tools, retrieval and agents.
LangGraph	Low-level stateful orchestration/runtime framework for workflows and agents.
LLM	Large Language Model.
Memory	Information retained for future interactions.
Node	A computational step in a graph.
Prompt	Instructions and context supplied to a model.
RAG	Retrieval-Augmented Generation; retrieve external evidence and provide it to a model.
ReAct	Pattern combining reasoning/decision-making with external actions and observations.
State	Data carried through a workflow execution.
Structured output	Model output constrained to a schema such as JSON.
Token	A unit of text processed by an LLM.
Tool	A callable external capability exposed to a model.
Vector store	Index/database optimised for similarity search over embeddings.

20 References and Further Reading

1. Vaswani et al. (2017), Attention Is All You Need. https://arxiv.org/abs/1706.03762

2. Yao et al. (2022/2023), ReAct: Synergizing Reasoning and Acting in Language Models. https://arxiv.org/abs/2210.03629

3. Schick et al. (2023), Toolformer: Language Models Can Teach Themselves to Use Tools. https://arxiv.org/abs/2302.04761

4. LangChain documentation - overview. https://docs.langchain.com/oss/python/langchain/overview

5. LangGraph documentation - overview. https://docs.langchain.com/oss/python/langgraph/overview

6. LangChain concepts - frameworks, runtimes and harnesses. https://docs.langchain.com/oss/python/concepts/products

7. LangChain tools documentation. https://docs.langchain.com/oss/python/langchain/tools

8. LangGraph interrupts documentation. https://docs.langchain.com/oss/python/langgraph/interrupts

9. LangGraph persistence documentation. https://docs.langchain.com/oss/python/langgraph/persistence

10. LangGraph workflows and agents. https://docs.langchain.com/oss/python/langgraph/workflows-agents

11. LangChain memory overview. https://docs.langchain.com/oss/python/concepts/memory

12. Groq supported models / Qwen 3.6 27B. https://console.groq.com/docs/model/qwen/qwen3.6-27b

13. Ollama tool calling. https://docs.ollama.com/capabilities/tool-calling

14. Ollama context length. https://docs.ollama.com/context-length

15. Ollama API introduction. https://docs.ollama.com/api/introduction

16. Ollama structured outputs. https://docs.ollama.com/capabilities/structured-outputs

17. Ollama thinking controls. https://docs.ollama.com/capabilities/thinking

18. NIST AI Risk Management Framework. https://www.nist.gov/itl/ai-risk-management-framework

19. NIST Generative AI Profile. https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence

20. Qwen3 Technical Report. https://arxiv.org/abs/2505.09388

20.1 Recommended sequence for further reading

1. Start with the LangChain and LangGraph overview pages [4-6] after completing Days 1-2.

2. Read ReAct [2] after you have built your first tool-using agent.

3. Read the LangGraph interrupts and persistence documentation [8-9] immediately before the HITL practical.

4. Read Ollama tool-calling and structured-output pages [13, 16] when experimenting with local models.

5. Use the NIST AI RMF resources [18-19] during the responsible-AI and capstone evaluation discussion.

6. Return to the Transformer paper [1] only after the beginner explanation of attention makes sense; it is a research paper, not an introductory tutorial.

20.2 Final reflection questions

1. What decisions should an LLM never be allowed to make alone in your chosen domain?

2. Which parts of an agent should be deterministic rather than model-driven?

3. What evidence would convince you that an agent is useful rather than merely impressive?

4. Where should the system stop and ask a human?

5. What information must be logged so another person can audit the agent later?

6. If you changed provider or model tomorrow, which parts of your architecture should remain unchanged?

21 Appendix A. Absolute-Beginner Python and Jupyter Primer

You do not need to be an expert programmer to follow the bootcamp, but you do need a small amount of Python literacy. This appendix explains the minimum concepts used repeatedly in the notebooks.

21.1 A.1 Variables and basic data types

```
name = "Aisha"          # string (text)
students = 24           # integer
latency = 1.42          # floating-point number
provider_ready = True   # Boolean

print(name, students, latency, provider_ready)
```

A variable is a name that refers to a value. Python infers the type from the assigned value. Strings use quotation marks; integers are whole numbers; floats contain decimal values; Booleans are True or False.

21.2 A.2 Lists and dictionaries

```
models = ["qwen3.5:0.8b", "llama3.2:1b"]

metadata = {
    "provider": "ollama",
    "model": "qwen3.5:0.8b",
    "latency_seconds": 2.1,
}

print(models[0])
print(metadata["provider"])
```

Lists store ordered collections. Dictionaries store key-value pairs. Agent state is very often represented with dictionary-like structures, so this pattern will appear frequently.

21.3 A.3 Functions

```
def throughput_gap(incoming, processed):
    return incoming - processed

gap = throughput_gap(120, 95)
print(gap)
```

A function packages reusable behaviour. In agent systems, ordinary Python functions often become tools.

21.4 A.4 Imports and packages

```
import os
from groq import Groq
from langgraph.graph import StateGraph
```

Python's standard library contains built-in modules such as os. Third-party packages such as groq and langgraph must be installed in the active Python environment before they can be imported.

21.5 A.5 Exceptions

```
try:
    result = 10 / 0
except ZeroDivisionError as exc:
    print("The operation failed:", exc)
```

An exception represents an error. Robust notebooks do not hide exceptions blindly; they catch expected failures and explain what the learner should check next.

21.6 A.6 What Jupyter actually does

A Jupyter notebook is a document connected to a running Python kernel. Each code cell sends Python code to the kernel. The kernel keeps variables in memory until it is restarted. This is why a notebook can appear to work after cells were run out of order but fail after "Restart Kernel and Run All". Reliable notebooks are written so that a clean top-to-bottom execution succeeds.

21.7 A.7 Notebook discipline

- Run setup/import cells first.
- Prefer "Restart Kernel and Run All" when testing reproducibility.
- Do not rely on variables created by an unrelated notebook.
- Read error messages from the bottom upward: the final line usually names the immediate exception.
- Record your active Python executable when debugging environment problems.

```
import sys
print(sys.executable)
print(sys.version)
```

22 Appendix B. APIs, HTTP and Environment Variables for Beginners

22.1 B.1 What is an API?

An API is a defined interface through which one software system requests a service from another. When you use Groq, your Python code sends an authenticated request across the internet. When you use Ollama locally, your code normally sends a request to a local server on your own machine.

22.2 B.2 Client and server

Role	Meaning	Example
Client	Software making the request	Your notebook
Server	Software receiving and processing it	Groq API or local Ollama service
Endpoint	Specific API address/function	Ollama /api/chat
Request	Data sent by client	Model + messages + options
Response	Data returned by server	Generated message + metadata

22.3 B.3 HTTP status codes

Range	Meaning	Example
2xx	Success	200 OK
4xx	Request/client/auth issue	401 unauthorised, 403 forbidden, 404 not found
429	Rate limit	Too many requests or tokens
5xx	Server-side failure	Temporary provider problem

Do not respond to every error by reinstalling packages. A 401/403 usually points to credentials or access; 404 may be an endpoint/model name; 429 is often quota/rate limiting; connection-refused against localhost usually means the local service is not listening.

22.4 B.4 Environment variables

Environment variables let software read configuration without embedding secrets directly in source code. This is the recommended pattern for API keys.

```
# macOS/Linux terminal
export GROQ_API_KEY="your_new_key_here"

# Python
import os
key = os.environ.get("GROQ_API_KEY")
```

22.5 B.5 The common os.environ.get mistake

```
# WRONG: the secret itself is treated as the variable NAME
os.environ.get("gsk_actual_secret_here")

# RIGHT: read the value stored under the variable name
os.environ.get("GROQ_API_KEY")
```

22.6 B.6 .env files

A .env file is convenient for local development, but it is still a secret-bearing file and should normally be excluded from Git. Distribute .env.example or .env.template with placeholder values, never the real key.

```
# .gitignore
.env
.venv/
__pycache__/
.ipynb_checkpoints/
```

23 Appendix C. Provider Setup: Groq and Ollama

23.1 C.1 Groq setup

1. Create a Groq account using the official Groq console.

2. Create a new API key in the API-key section.

3. Copy it once into a secure password manager or local environment configuration.

4. Set GROQ_API_KEY in your shell or .env file.

5. Install the SDK with pip install groq.

6. Run a minimal SDK call before introducing LangChain or LangGraph.

Official documentation: https://console.groq.com/docs

```
from groq import Groq
import os

client = Groq(api_key=os.environ["GROQ_API_KEY"])
models = client.models.list()
print([m.id for m in models.data][:10])
```

23.2 C.2 Ollama setup

1. Install Ollama from the official website for your operating system.

2. Start Ollama.

3. Open a terminal and run ollama list.

4. If necessary, pull a model with ollama pull <model>.

5. Verify the local server before running LangChain/LangGraph integration.

Official documentation: https://docs.ollama.com

```
# Terminal
ollama list
ollama ps

# Basic local HTTP check
curl http://127.0.0.1:11434/api/tags
```

23.3 C.3 Why an installed model can still “fail”

- The Ollama service may not be running.
- The notebook may be trying localhost/host settings that differ from your configuration.
- The exact model name may differ in capitalisation or tag.
- The model may not support the requested capability such as tool calling.
- The request may use a context size or model too large for available memory.
- The Python environment running Jupyter may not contain the ollama package even though the command-line app is installed.

23.4 C.4 Provider metadata students should report

Field	Why record it
Provider	Reproducibility
Model ID	Models differ significantly
Endpoint/host	Distinguishes local from hosted execution
Latency	User experience/performance
Input/output tokens	Cost and efficiency
Finish reason	Helps detect truncation
Reasoning/thinking mode	Can alter latency and output
Timestamp	Useful when services/models change

24 Appendix D. Debugging Agentic AI Notebooks Systematically

24.1 D.1 Use a layer-by-layer diagnostic strategy

1. Confirm Python and notebook kernel.

2. Confirm package imports.

3. Confirm credentials or local service.

4. Confirm provider/model with a minimal call.

5. Confirm tool independently without the LLM.

6. Confirm graph nodes independently.

7. Run the complete agent last.

This strategy prevents a LangGraph error from being misdiagnosed as an Ollama error, or a credential error from being hidden inside a complicated agent loop.

24.2 D.2 Minimal provider test before frameworks

If the raw Groq SDK call fails, do not debug LangChain yet. If the raw Ollama Python or HTTP call fails, do not debug LangGraph yet. Reduce the system until the smallest failing layer is isolated.

24.3 D.3 Progress indicators

A notebook should communicate that work is ongoing. For a single blocking request, print the current stage and elapsed time or run a lightweight spinner in another thread. For loops over documents or evaluations, use tqdm. For graphs, emit node names or use streaming so the learner can see transitions.

```
from tqdm.auto import tqdm

results = []
for item in tqdm(items, desc="Evaluating cases"):
    results.append(run_case(item))
```

24.4 D.4 Timeouts and maximum steps

Any network call or agent loop can stall. Set reasonable request timeouts and maximum iterations. A timeout should produce an actionable message stating provider, model, elapsed time and the next diagnostic command.

24.5 D.5 Common error matrix

Symptom	Likely layer	First check
ModuleNotFoundError	Python environment	sys.executable + pip install
401/403	Credential/access	Key and provider account

Symptom	Likely layer	First check
429	Quota/rate limit	Provider limits and retry-after
Connection refused 11434	Ollama service	ollama list / curl local endpoint
Model not found	Model ID/tag	Provider model list / ollama list
Graph KeyError	State schema	Node input/output keys
Agent loops forever	Control logic	Max steps + stopping condition
Blank answer	Provider/model or parsing	Raw response + finish reason

25 Appendix E. Reusable Agent Engineering Patterns

25.1 E.1 “LLM where useful, code where exact”

Use LLMs for ambiguity, language understanding, classification, synthesis and bounded planning. Use ordinary code for arithmetic, schema validation, permissions, deterministic business rules and irreversible side effects. This division makes systems easier to test and govern.

25.2 E.2 Read-analyse-propose-approve-act

A safe professional pattern is to let the agent read evidence and perform analysis, then require a human to approve a proposed action before a write tool executes. This creates a natural audit boundary.

25.3 E.3 Evidence before recommendation

Require the agent to populate an evidence field before generating a recommendation. If evidence is insufficient, route to clarification or retrieval rather than allowing the model to fill gaps by speculation.

25.4 E.4 Confidence is not certainty

A model-generated confidence score is not a calibrated probability by default. Treat it as one signal. Prefer objective checks such as evidence coverage, tool success, schema validation and human review for important tasks.

25.5 E.5 Idempotency

An idempotent write can be safely repeated without producing duplicate side effects. For external actions, use operation IDs or deduplication keys so a retry does not send the same email twice or create duplicate records.

26 Appendix F. Teaching Questions, Misconceptions and Discussion Prompts

26.1 F.1 Common misconceptions to surface explicitly

- “An LLM searches the internet automatically.” - Only if a tool or provider capability gives it web access.
- “RAG trains the model on my documents.” - No; RAG retrieves documents at inference time.
- “LangChain is an LLM.” - No; it is an application framework.
- “LangGraph is only for multiple agents.” - No; it is useful for any stateful graph/workflow.

- "HITL means showing the final answer to a person." - Genuine HITL can pause before consequential action and resume with the human decision.
- "A local model is automatically private." - Local processing helps, but logging, telemetry, downloaded data and application design still matter.
- "Bigger model always means better system." - Architecture, evidence, tools, latency and evaluation matter too.

26.2 F.2 Socratic discussion prompts

1. When should an organisation prefer a deterministic workflow over an agent?

2. What is the minimum autonomy necessary for this task?

3. Which tool should be read-only, and which should require approval?

4. What evidence would a reviewer need before approving an action?

5. Could the same objective be achieved without an LLM?

6. What happens if a retrieved document is malicious?

7. How would you prove that switching models did not break the system?

8. Which metrics matter most: accuracy, latency, cost, user effort or safety?

27 Appendix G. A Small End-to-End Architecture Walkthrough

Consider a teaching agent that investigates a hypothetical rise in fulfilment backlog. The objective is not to control a real facility. It is to demonstrate how a professional architecture separates evidence retrieval, calculation, model reasoning and human authority.

27.1 G.1 Proposed state

```
class OpsState(TypedDict):
    request: str
    metrics: dict
    retrieved_guidance: list[str]
    throughput_gap: float
    risk_level: str
    proposal: str
    human_decision: dict | None
    final_response: str
```

27.2 G.2 Nodes

Node	Responsibility
classify	Identify request type and risk indicators
retrieve_metrics	Call read-only operational data tool

Node	Responsibility
retrieve_guidance	Retrieve approved guidance
calculate	Use deterministic calculator
draft	LLM synthesises evidence into proposal
approval	Interrupt if proposal is consequential
finalise	Apply decision and produce response

27.3 G.3 Why this is better than one giant prompt

- Each step can be inspected and tested.
- Exact calculations are delegated to code.
- Evidence and recommendations are separated.
- The human sees the proposal before action.
- Failures can be traced to a node or tool.
- Provider/model can change without redesigning the whole workflow.